SOFTWARE ENGINEERING PROJECT

MeetRX

BY

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY

Academic Year 2026

MeetRX

BY

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

A Project Submitted in Partial Fulfillment of the Requirement for the Degree of Bachelor of Engineering (Software and Knowledge Engineering)
Department of Computer Engineering
Faculty of Engineering
KASERTSART UNIVERSITY
Academic Year 2026

Approved By:

Advisor . . . . . **Date** . . . . . .
(Project Advisor's Name)

Head of Department . . . . . **Date** . . . . . .
(Head of Department's Name)

Abstract

Put your abstract paragraph here.

Acknowledgement

Put your acknowledgement paragraph here.

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

Table of Contents

Content **Page**

Abstract **i**

Acknowledgement **ii**

List of Tables **v**

List of Figures **vi**

Chapter 1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Solution Overview 2
1.3.1 Features 3
1.4 Target User 3
1.5 Terminology 3

Chapter 2 Literature Review and Related Work **5**
2.1 Competitor Analysis 5
2.2 Literature Review 6
2.2.1 Real-Time Speech Analysis 6
2.2.2 Meeting Summarization 6
2.2.3 Task Allocation 7
2.2.4 Topic Drift Detection 7
2.2.5 Agentic AI and Code Analysis 8
2.3 Summary 8

Chapter 3 Requirement Analysis **9**
3.1 Stakeholder Analysis 9
3.2 User Stories and Use Cases 9
3.2.1 Student 9
3.2.2 Software Developer 10
3.2.3 Scrum Master 10
3.3 Use Case Diagrams 11
3.4 Use Case Model 12
3.5 User Interface Design 13

Chapter 4 Software Architecture Design **15**
4.1 System Architecture Overview 15
4.2 Software Design 15
4.3 AI and Data Design 15
4.3.1 Data Sources and Collection 15
4.3.2 Model Design 16
4.3.3 Evaluation 16

Chapter 5 Software Development **17**
5.1 Schedule 17

Appendix A: Example **22**

Appendix B: About LaTeX **24**

List of Tables

Page

List of Figures

Page

2.1 Competitive Analysis Table 5
3.1 Use Case Diagram 11
3.2 User Interface Wireframe Design 14

Chapter 1

Introduction

1.1 Background

Meetings are an indispensable tool for collaborative work in both academic and professional settings. In theory, a focused meeting covering a single topic should require no more than 15–30 minutes. In practice, however, teams routinely turn short discussions into multi-hour sessions. The root cause is *topic drift*: the gradual, often unnoticed, departure of conversation from the original agenda.

The development team behind MeetRX experienced this problem first-hand during group project work. What should have been a concise weekly sync repeatedly stretched into a three-hour session because members lost track of the intended focus and the conversation wandered across unrelated topics. This observation motivated the creation of an intelligent meeting assistant that can detect drift as it occurs and help teams stay aligned.

1.2 Problem Statement

The following problems were identified through the team's own experience and stakeholder interviews:

1. **Burden on the minute-taker.** During meetings, the person responsible for taking notes cannot participate meaningfully in the discussion. Their attention is divided between listening, comprehending, and writing, which reduces both the quality of the notes and their contribution to the meeting.

2. **Standup meetings overrunning their allotted time.** Daily standups and progress-update meetings are intended to be short, time-boxed events. In practice, they frequently exceed their scheduled duration because participants introduce tangential topics, triggering lengthy discussions that should be handled separately.

3. **Repeated debates caused by poor institutional memory.** In long-term projects with frequent meetings, teams often forget what was discussed or decided in earlier sessions. As a result, the same topics are re-opened and the same decisions are re-debated, consuming time that could be spent on new work.

1.3 Solution Overview

MeetRX addresses the problems above through three integrated AI subsystems:

1. **Summarisation AI**: Automatically transcribes meeting audio using OpenAI Whisper and generates a structured summary so that no human minute-taker is required. All participants can engage fully in the discussion.

2. **Topic Drift Detection AI (Main Feature)**: Continuously analyses the live conversation using natural language processing techniques to assign a drift score to recent utterances. When the score exceeds a configurable threshold, a large language model re-evaluation is triggered and a real-time alert is surfaced to meeting participants, prompting them to return to the agenda.

3. **Agentic AI**: Provides a conversational interface that can answer questions about past meetings, look up decisions recorded in previous summaries, query the project codebase, and integrate with external tools to retrieve relevant context on demand.

1.3.1 Features

1. **Real-time Topic Drift Detection**: Monitors conversation flow and raises an alert when discussion deviates from the stated agenda topic.
2. **Automatic Meeting Summarisation**: Produces a concise, structured summary at the end of every session without requiring manual note-taking.
3. **Q&A Chatbot**: Allows participants to query past meeting summaries and project context using natural language.
4. **Task Assignment**: Extracts and records action items from meeting transcripts and assigns them to the appropriate team members.

1.4 Target User

MeetRX is designed for three primary user groups:

- **Students**: Particularly those conducting group project meetings who need help keeping discussions on-topic and producing reliable records of decisions made.
- **Software Developers**: Engineering teams running daily standups, sprint planning, and retrospectives who want shorter, more focused meetings and automatic summaries.
- **Scrum Masters**: Facilitators responsible for sprint ceremonies who need tooling to enforce time-boxes and surface relevant historical context during discussions.

1.5 Terminology

Topic Drift A situation in which the ongoing conversation is no longer contributing to the intended meeting topic or agenda item.

Drift Score A numerical value computed by the NLP pipeline that quantifies how far the current utterance has deviated from the baseline topic embedding.

Agenda A predefined list of topics that a meeting is intended to cover.

Utterance A single spoken turn or message produced by one participant during a meeting.

Summarisation The process of condensing a meeting transcript into a shorter document that captures the key points, decisions, and action items.

Agentic AI An AI system capable of autonomously invoking external tools or data sources to answer queries without explicit step-by-step instructions from the user.

LLM Large Language Model; a neural network trained on large text corpora capable of generating and understanding natural language.

Whisper An open-source automatic speech recognition model developed by OpenAI.

Chapter 2

Literature Review and Related Work

2.1 Competitor Analysis

Name	Meeting Summary	Q&A Chatbot	Task Assignment	Topic Drift Detection
Otter AI	✓	✓	✗	✗
Fireflies AI	✓	✓	✓ Only in its own Project Management software	✗
Spinach AI	✓	✓	✓	✗
MeetRX	✓	✓	✓	✓

Figure 2.1: Competitive Analysis Table

Otter AI provides real-time transcription, automatic meeting summaries, and a Q&A interface for querying transcripts. However, it has no task assignment capability and no mechanism for detecting or alerting users to topic drift.

Fireflies AI supports transcription, summarisation, and a chatbot interface. Its task assignment feature is limited to its own proprietary project management integration and does not support arbitrary third-party tools. Topic drift detection is absent.

Spinach AI is targeted at agile teams and offers summaries, Q&A, and task assignment with broader tool integration. Like the others, it provides no real-time topic drift detection.

MeetRX is the only solution in this comparison that addresses all four dimensions, and is the only product that actively detects and alerts participants to topic drift during a live meeting.

2.2 Literature Review

This section reviews academic research and technical reports relevant to the design and implementation of MeetRX, focusing on four key areas: real-time transcription, meeting summarization, task allocation, topic drift detection, and agentic AI systems. These works provide the technical foundation for the design of MeetRX and how it aids in selecting appropriate metrics for evaluating the system's performance.

2.2.1 Real-Time Speech Analysis

Real-time speech transcription is a fundamental component of the system, enabling the automatic conversion of meeting audio into text. The study in[1] explores the integration of NLP with VoIP systems to achieve low-latency transcription.

Another work[2] propose to use Recurrent Neural Network Transducers models to improve accuracy and speed of transcription in noisy environments, which is crucial for real-world meeting scenarios.

To evaluate the performance of the transcription module, we will use metrics such as Word Error Rate (WER)[3] and latency. WER measures the accuracy of the transcription by comparing the transcribed text to a reference transcript, while latency assesses the time taken from audio input to text output.

2.2.2 Meeting Summarization

Meeting summarization aims to shorten and condense the content of a meeting transcript while preserving key information. The MISeD dataset[4] introduce a method for generating dialogue data, particularly for meeting scenarios, the QMSum dataset[5] provides annotated meeting transcripts designed for query-based summarization tasks.

Evaluating summarization quality can be done using ROUGE[6] and BERTScore[7], which measure the overlap of n-grams and semantic similarity between the generated summary and reference summaries.

2.2.3 Task Allocation

Task allocation is a critical component of team productivity in agile environments. The study in[8] explores the use of machine learning algorithms to assign tasks in distributed software development teams based on historical performance and expertise.

Similarly, the TaskAllocator framework[9] proposes a recommendation-based approach that assigns tasks according to roles and past project data.

These approaches highlight the potential of data-driven task assignment to reduce manual coordination and improve efficiency. By learning from historical patterns, systems can make informed decisions about task distribution from the available data.

Inspired by these works, MeetRX decides to implement a task assignment feature that extracts action items from meeting transcripts and assigns them to appropriate team members based on expertise or roles.

2.2.4 Topic Drift Detection

Topic drift detection is the core feature for MeetRX, enabling real-time alerts when a meeting conversation drifts away from the agenda. The TIAGE framework[10] introduces a methods for detecting changes in dialog using topic aware generation (GEN Encoder).

Another study[11] focuses on detecting topic shifts in mixed-initiative conversations, highlighting the importance of context in conversational AI systems.

These studies suggest that semantic similarity and contextual embeddings are effective for identifying topic drift.

Based on these findings, MeetRX will computes drift score using the embedding similarity between the current conversation and the predefined agenda. If the score exceeds a certain threshold, an alert will be triggered to notify participants of potential topic drift.

2.2.5 Agentic AI and Code Analysis

Recent advancement in agentic AI enables systems to perform complex tasks autonomously. Frameworks such as those described in[12] demonstrate how LLMs can be integrated with toolchains to analyze code repositories and provide insights on the user queries.

In addition, the agentic AI system for the platform such as GitHub's Agentic Workflows[13] and agentic code review systems[14] show how AI can assist in automating software development tasks, including code analysis and review.

Evaluating the performance of agentic AI systems involves metrics such as task success rate, response accuracy, and tool usage efficiency[15]. These metrics ensure that the system produces reliable and useful outputs.

Based on these findings, MeetRX integrates an agentic AI component that allows users to query past meeting information to retrieve relevant details.

2.3 Summary

This chapter has reviewed the competitive landscape of meeting productivity tools and the relevant academic research that informs the design of MeetRX. The analysis highlights the unique features of MeetRX, particularly its real-time topic drift detection, which sets it apart from existing solutions. The literature review provides a solid foundation for the technical implementation of MeetRX and guides the selection of appropriate evaluation metrics for assessing its performance in subsequent chapters.

Chapter 3

Requirement Analysis

3.1 Stakeholder Analysis

Students University students working in group projects need a tool that detects when their meeting drifts off the project agenda and automatically generates meeting minutes so that all members can participate in the discussion rather than taking turns as note-taker.

Software Developers (Office Workers) Software engineers attending daily standups and sprint ceremonies benefit from automatic meeting summaries and from an agent that can surface relevant decisions from past sessions, reducing repeated debate.

Scrum Masters / Project Managers Facilitators need visibility into whether a meeting is staying on-topic and access to a historical record of decisions so they can run tighter, more accountable ceremonies.

3.2 User Stories and Use Cases

3.2.1 Student

1. As a **student**, I want the system to alert me when my group's conversation goes off-topic, so that we can finish our meeting within the scheduled time.
2. As a **student**, I want an automatic summary of each meeting, so that I do not have to assign a dedicated note-taker.
3. As a **student**, I want to query what was decided in a previous meeting, so that I do not have to re-read full transcripts.

3.2.2 Software Developer

1. As a **software developer**, I want standup meetings to stay focused on project progress, so that they finish within 15 minutes.

2. As a **software developer**, I want action items extracted automatically from the meeting transcript and assigned to the correct team member, so that nothing falls through the cracks.

3. As a **software developer**, I want to ask the AI agent about a past architectural decision, so that I do not have to search through old chat logs or documents.

3.2.3 Scrum Master

1. As a **scrum master**, I want a real-time indicator that shows whether the current discussion is on-topic, so that I can redirect the team without interrupting the conversation flow.

2. As a **scrum master**, I want all sprint summaries stored and indexed, so that I can reference them during retrospectives.

3.3 Use Case Diagrams

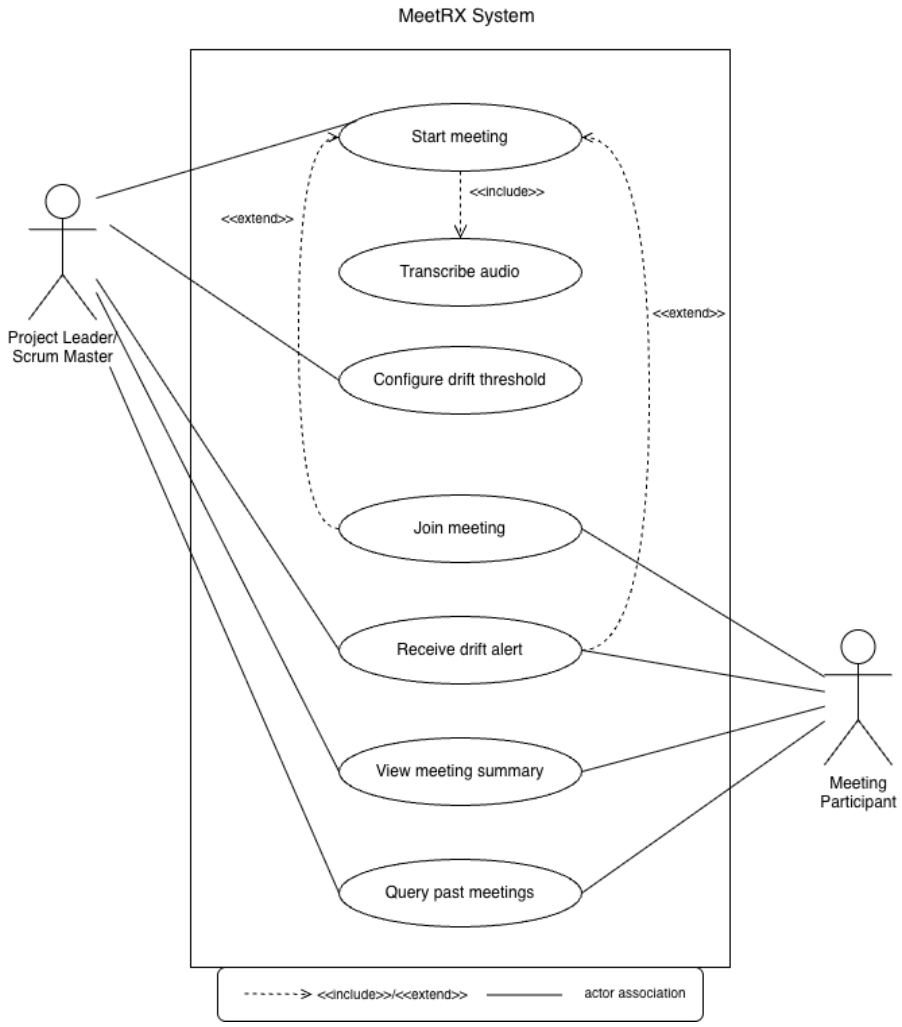

Figure 3.1: Use Case Diagram

3.4 Use Case Model

UC-01: Start Meeting

Actor	Meeting Participant
Precondition	The participant is authenticated and has a meeting agenda prepared.
Main Flow	1. Participant opens the MeetRX dashboard. 2. Participant clicks “Start Meeting” and enters agenda topics. 3. System begins audio capture and topic embedding initialisation. 4. Meeting session is created and all participants are notified.
Postcondition	An active meeting session exists with an initialised topic baseline.

UC-02: Receive Topic Drift Alert

Actor	Meeting Participant (all)
Precondition	A meeting session is active.
Main Flow	1. NLP module computes drift score for each utterance. 2. If score exceeds threshold, LLM re-evaluation is triggered. 3. System displays a drift alert banner visible to all participants. 4. Participant acknowledges the alert and guides conversation back to agenda.
Alternative Flow	If the participant dismisses the alert, the system logs the dismissal and resumes monitoring.
Postcondition	Alert is logged; meeting continues.

UC-03: Query Past Meetings

Actor	Meeting Participant
Precondition	At least one completed meeting summary exists in the system.
Main Flow	1. Participant opens the Q&A interface. 2. Participant types a natural language question. 3. Agentic AI retrieves relevant summaries and generates an answer. 4. System displays the answer with source meeting references.
Postcondition	Participant receives an answer grounded in historical meeting data.

3.5 User Interface Design

Figure 3.2: User Interface Wireframe Design

Chapter 4

Software Architecture Design

4.1 System Architecture Overview

<TIP: Provide a high-level description of your system's structure. Include either a Domain Model (to describe core business concepts and their relationships) or a Design Class Diagram (to describe how your code is organized into classes and packages), whichever best communicates your design. Accompany your diagram with a short paragraph explaining the key components and how they interact. />

4.2 Software Design

<TIP: Describe the runtime behavior of your system. Include a Sequence Diagram for your most important use cases. Focus on scenarios with non-trivial interactions between components. If your project involves a novel algorithm, describe it here and explain why you designed it this way. />

4.3 AI and Data Design

4.3.1 Data Sources and Collection

<TIP: Describe where your data comes from. Is it collected from users, scraped from the web, sourced from a public dataset, or provided by an external API? Explain how much data you have or plan to collect, and any challenges you anticipate in obtaining it. Explain how data is cleaned, transformed, or prepared before use. />

4.3.2 Model Design

<TIP: Describe the AI model or external AI service you plan to use. Explain why you chose it over alternatives. This can connect back to your competitor or literature review in Chapter 2. If you are training your own model, describe the architecture and training approach. If you are using a third-party model or API, describe how it is integrated into your system and any prompt design or fine-tuning involved. />

4.3.3 Evaluation

<TIP: Describe the metrics you plan to use to evaluate your AI model. />

Chapter 5

Software Development

5.1 Schedule

<TIP: Show the schedule of your project. Use a Gantt chart. The items should agree with features you listed in Chapter 1. />

Reference

Bibliography

[1] K. Rajchandar, P. N. Hegde, U. Dwivedi, K. Chouhan, K. S. Sidhu, and R. Velagala, "Natural language processing for real-time transcription in voip systems," in *Proceedings of the 2025 International Conference on Computing Technologies and Data Communication (ICCTDC)*, 2025, pp. 1–5.

[2] J. Ma and Y. Liu, "Research on real-time speech transcription optimization based on recurrent neural network transducer," in *Proceedings of the 2023 IEEE International Conference on Image Processing and Computer Applications (ICIPCA)*, 2023, pp. 1571–1576.

[3] T. P. Code, "Calculate word error rate (wer) in python," 2023. [Online]. Available: https://thepythoncode.com/article/calculate-word-error-rate-in-python

[4] "Efficient data generation for source-grounded information-seeking dialogs: A use case for meeting transcripts," *arXiv*, 2024. [Online]. Available: https://arxiv.org/pdf/2405.01121

[5] Y. LILY, "Qmsum dataset," 2021. [Online]. Available: https://github.com/Yale-LILY/QMSum

[6] Santhosh, "Understanding bleu and rouge score for nlp evaluation," 2023. [Online]. Available: https://medium.com/@sthanikamsanthosh1994/understanding-bleu-and-rouge-score-for-nlp-evaluation

[7] Abonia, "Bertscore explained in 5 minutes," 2023. [Online]. Available: https://medium.com/@abonia/bertscore-explained-in-5-minutes

[8] "Utilizing machine learning algorithms for task allocation in distributed agile software development," *PMC*, 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11567032/

[9] "Taskallocator: A recommendation approach for role-based tasks allocation in agile software development," *arXiv*, 2021. [Online]. Available: https://arxiv.org/pdf/2103.02330

[10] "Tiage: Topic-aware dialogue generation," *arXiv*, 2021. [Online]. Available: https://arxiv.org/pdf/2109.04562

[11] "Topic shift detection for mixed initiative response," *ACL Anthology*, 2021. [Online]. Available: https://aclanthology.org/2021.sigdial-1.17.pdf

[12] ActiveWizards, "Build an ai agent for github code analysis with langchain," 2024. [Online]. Available: https://activewizards.com/blog/build-an-ai-agent-for-github-code-analysis-with-langchain

[13] GitHub, "Github agentic workflows," 2024. [Online]. Available: https://github.github.com/gh-aw/

[14] CyberArk, "Agentic code review demo," 2024. [Online]. Available: https://github.com/cyberark/agentic-code-review-demo

[15] GPTalk, "How to evaluate agentic ai: Key metrics and real-world examples," 2024. [Online]. Available: https://medium.com/gptalk/how-do-you-evaluate-agentic-ai-key-metrics-real-world-examples-781004de0280

[16] "Learn latex in 30 minutes," https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes, Overleaf.

Appendix A

Appendix A: Example

<TIP: Put additional or supplementary information/data/figures in appendices. />

Appendix B

Appendix B: About LaTeX

LaTeX (stylized as LaTeX) is a software system for typesetting documents. LaTeX markup describes the content and layout of the document, as opposed to the formatted text found in WYSIWYG word processors like Google Docs, LibreOffice Writer, and Microsoft Word. The writer uses markup tagging conventions to define the general structure of a document, to stylize text throughout a document (such as bold and italics), and to add citations and cross-references.

LaTeX is widely used in academia for the communication and publication of scientific documents and technical note-taking in many fields, owing partially to its support for complex mathematical notation. It also has a prominent role in the preparation and publication of books and articles that contain complex multilingual materials, such as Arabic and Greek.

Overleaf has also provided a 30-minute guide on how you can get started on using LaTeX. [16]